

IA-REFLEXION-C: El equipo descubrió que el invariante es un contrato lógico inquebrantable que colapsa si no podemos descartar mitades con seguridad. La condición absoluta que garantiza la corrección del algoritmo, sea en su versión iterativa o recursiva.

IA-REFLEXION-A: El equipo mostró confusión sobre si usar `izq < der` o `izq <= der` y cómo actualizar los límites para evitar bucles infinitos. Lograron implementar la búsqueda lineal de forma autónoma y entendieron bien el concepto de expansión de duplicados.

""

Semana 11 — Búsqueda Lineal, Binaria y HITO 2

Estructura de Datos Básica · UAN · LSC

Dr. Eligardo Cruz Sánchez

""

```
import time
import random
import copy
import sys
```

#

EVIDENCIAS DE COMPRENSIÓN (llenar ANTES de codificar)

#

COMPRENDE-1: Enuncia el invariante de búsqueda binaria con tus palabras.

RESPUESTA: Si el objetivo está en el arreglo, siempre se encuentra dentro del rango

[izq, der]. Cada paso reduce este rango a la mitad sin romper esta regla.

COMPRENDE-2: ¿Cuántas comparaciones hace búsqueda binaria en el peor caso para $n = 1\,000\,000$?

RESPUESTA: Hace 20 comparaciones ($\log_2(1000000) = 19.93 \rightarrow$ truncado a $19 + 1 = 20$).

COMPRENDE-3: ¿Por qué búsqueda binaria requiere arreglo ordenado?

RESPUESTA: Porque se basa en la propiedad transitiva: si el valor en 'mid' es menor

que mi objetivo, todo lo que está a la izquierda también lo es y puedo descartarlo.

Si el arreglo estuviera desordenado, descartaríamos posibles respuestas correctas.

ESTRATEGIA: Describe brevemente el plan del equipo para el HITO 2

RESPUESTA: Utilizaremos Merge Sort porque garantiza un tiempo $O(n \log n)$ en el peor

de los casos y es un algoritmo estable, lo cual es útil si ordenamos elementos

complejos (como tuplas del catálogo).

#

IMPORTAR CÓDIGO DE S9 Y S10

#

```
def merge_sort(A):
    if len(A) <= 1:
        return A
    mid = len(A) // 2
    izq = merge_sort(A[:mid])
    der = merge_sort(A[mid:])
    return merge(izq, der)

def merge(izq, der):
    resultado = []
    i = j = 0
    while i < len(izq) and j < len(der):
        # Comparamos por el puntaje (índice 0)
        if izq[i][0] <= der[j][0]:
            resultado.append(izq[i])
            i += 1
        else:
            resultado.append(der[j])
            j += 1
    resultado.extend(izq[i:])
    resultado.extend(der[j:])
    return resultado
```

#

SECCIÓN 1 — BÚSQUEDA LINEAL

#

```
def busqueda_lineal(A, objetivo):
    # DECISIÓN: ¿Por qué iterar con enumerate en lugar de range?
    # RESPUESTA: Porque 'enumerate' extrae directamente el valor y su posición (índice)
    # de memoria en un solo paso, siendo más legible y eficiente en Python que usar A[i].

    for i, valor in enumerate(A):
        if valor == objetivo:
            return i
    return -1
```

#

SECCIÓN 2 — BÚSQUEDA BINARIA (iterativa)

#

```
def busqueda_binaria(A, objetivo):
    # DECISIÓN: ¿Por qué usar mid = (izq + der) // 2 y no (izq + der) / 2?
    # RESPUESTA: Porque los índices de los arreglos deben ser números enteros.
    # El operador // realiza división entera (trunca el decimal).

    izq = 0
    der = len(A) - 1

    # Usamos <= para no omitir el caso donde el rango se reduce a 1 elemento.
    while izq <= der:
        mid = izq + (der - izq) // 2

        if A[mid] == objetivo:
            return mid
        elif A[mid] < objetivo:
            izq = mid + 1 # +1 porque ya descartamos la posición 'mid'
        else:
            der = mid - 1 # -1 porque ya descartamos la posición 'mid'

    return -1
```

#

SECCIÓN 3 — BENCHMARK COMPARATIVO

#

```
def generar_arreglo_ordenado(n):
    return list(range(n))

def medir_busqueda(funcion_busqueda, A, objetivo):
    inicio = time.perf_counter()
    resultado = funcion_busqueda(A, objetivo)
    fin = time.perf_counter()
    return fin - inicio, resultado

def ejecutar_benchmark_busqueda():
    tamanos = [1_000, 10_000, 100_000, 1_000_000]
    escenarios = [
        ('inicio', lambda n: 0),
        ('final', lambda n: n - 1),
        ('no_existe', lambda n: n + 999),
    ]

    print("=" * 90)
```

```
print(f'{'n':>10} | {'Escenario':>10} | {'Lineal (s)':>13} | {'Binaria (s)':>13} | {'Factor':>8}')
print("=" * 90)
```

for n in tamanos:

 A = generar_arreglo_ordenado(n)

 for nombre_esc, get_objetivo in escenarios:

 objetivo = get_objetivo(n)

 t_lin, res_lin = medir_busqueda(busqueda_lineal, A, objetivo)

 t_bin, res_bin = medir_busqueda(busqueda_binaria, A, objetivo)

 assert res_lin == res_bin, "DISCREPANCIA"

 factor = t_lin / t_bin if t_bin > 0 else float("inf")

 print(f'{'n':>10} | {'nombre_esc':>10} | {'t_lin':>13.6f} | {'t_bin':>13.6f} | {'factor':>7.1f}x")

 print("-" * 90)

REFLEXIONA-1: ¿En qué escenario la diferencia de velocidad es mayor?

RESPUESTA: En 'final' y 'no_existe'. La búsqueda lineal tiene que revisar

todo el millón de registros, tardando mucho más frente a los 20 pasos de binaria.

REFLEXIONA-2: ¿El "factor" de velocidad corresponde al ratio $O(n)/O(\log n)$?

RESPUESTA: Sí, teóricamente para $1,000,000 / 20 = 50,000$. Los datos empíricos

del factor x muestran este salto brutal de aceleración.

REFLEXIONA-3: ¿Qué pasa si buscan el objetivo al INICIO con búsqueda lineal vs binaria?

RESPUESTA: La lineal es instantánea ($O(1)$) porque está en la posición 0.

La binaria tardará más porque deberá partir el arreglo por la mitad hasta llegar al 0.

def ejecutar_benchmark_duplicados():

 print("\n" + "=" * 75)

 print("BENCHMARK DUPLICADOS MASIVOS")

 print("=" * 75)

REFLEXIONA-DUPLICADOS: ¿Cómo cambia el tiempo conforme aumenta el % de duplicados?

RESPUESTA: El tiempo deja de ser $O(\log n)$ y se vuelve drásticamente más lento.

Con 50% o 90% de duplicados, la "expansión lineal" arruina el esfuerzo logarítmico,

porque terminas recorriendo a pie miles de registros ($O(k)$, donde k son los duplicados).

#

¿Qué función del Reto 6 resolvería esto?

Buscar_rango. En lugar de expandirse linealmente, usaría dos búsquedas binarias:

una para hallar la primera ocurrencia y otra para la última, manteniendo todo $O(\log n)$.

#

SECCIÓN 4 — PIPELINE HITO 2

#

```

catalogo_streamx = [
    (8.2, "Interestelar", "MX-001"),
    (7.5, "El Laberinto", "MX-002"),
    (9.1, "Origen", "MX-003"),
    (6.8, "Dune Parte 1", "MX-004"),
    (8.7, "Oppenheimer", "MX-005"),
    (7.5, "La Señal", "MX-006"),
    (9.4, "Everything E.E.", "MX-007"),
    (6.2, "Los Elegidos", "MX-008"),
    (8.0, "The Batman", "MX-009"),
    (7.5, "Gravedad", "MX-010"),
    (9.0, "Parasite", "MX-011"),
    (7.1, "Midsommar", "MX-012"),
]

```

```

def ordenar_catalogo(catalogo, algoritmo='veredicto'):
    # DECISIÓN: Justificar la elección del algoritmo.
    # RESPUESTA: Usamos Merge Sort ('veredicto') por su complejidad  $O(n \log n)$  asegurada
    # en cualquier caso y por ser estable, evitando degradaciones catastróficas.
    copia = copy.deepcopy(catalogo)
    if algoritmo in ('veredicto', 'merge_sort'):
        return merge_sort(copia)
    return copia

```

```

def buscar_por_puntaje(catalogo_ordenado, puntaje_objetivo):
    # Extraemos solo los puntajes para pasarlos a nuestra búsqueda binaria
    puntajes = [pelicula[0] for pelicula in catalogo_ordenado]
    idx = busqueda_binaria(puntajes, puntaje_objetivo)

    if idx == -1:
        return []

    resultados = []

    # Expandir hacia la izquierda
    i = idx
    while i >= 0 and catalogo_ordenado[i][0] == puntaje_objetivo:
        resultados.append(catalogo_ordenado[i])
        i -= 1

    # Reordenar para mantener el sentido original tras iterar hacia atrás
    resultados.reverse()

    # Expandir hacia la derecha (saltando 'idx' para no duplicarlo)
    j = idx + 1
    while j < len(catalogo_ordenado) and catalogo_ordenado[j][0] == puntaje_objetivo:
        resultados.append(catalogo_ordenado[j])
        j += 1

    return resultados

```

```

def pipeline_hito2(catalogo, puntaje_buscar, algoritmo='veredicto'):

```

```

inicio_orden = time.perf_counter()
catalogo_ordenado = ordenar_catalogo(catalogo, algoritmo)
t_orden = time.perf_counter() - inicio_orden

inicio_busqueda = time.perf_counter()
resultados = buscar_por_puntaje(catalogo_ordenado, puntaje_buscar)
t_busqueda = time.perf_counter() - inicio_busqueda

return {
    'catalogo_ordenado': catalogo_ordenado,
    'resultados': resultados,
    't_orden_s': t_orden,
    't_busqueda_s': t_busqueda,
    't_total_s': t_orden + t_busqueda,
}

if __name__ == "__main__":
    resultado = pipeline_hito2(catalogo_streamx, puntaje_buscar=7.5)
    print(f"Catálogo ordenado (primeros 5): {resultado['catalogo_ordenado'][:5]}")
    print(f"\nPelículas con puntaje 7.5: {resultado['resultados']}")
    print(f"Tiempo total: {resultado['t_total_s']:.6f} s")

# VALIDA: ¿El pipeline retorna correctamente las 3 películas con puntaje 7.5?
# RESPUESTA: Sí, el algoritmo retornó exitosamente las tres películas:
# "El Laberinto", "La Señal" y "Gravedad", cumpliendo con el requerimiento
# del Hito 2 para la Ing. Sofía.

```

IA-REFLEXION-B: El equipo comprendió que evitar el retorno inmediato y seguir acotando el espacio permite encontrar los límites exactos del rango. Entendieron que el tiempo $O(\log n)$ se mantiene porque seguimos dividiendo el arreglo a la mitad en cada iteración, sin importar la cantidad de duplicados.

IA-REFLEXION-R: El ítem D1 (trazabilidad) fue el más difícil, evidenciando debilidad en la ejecución técnica de la prueba de escritorio. Antes de la defensa real, el equipo necesita repasar el seguimiento manual y exacto de los índices `izq`, `der` y `mid` iteración por iteración.

IA-REFLEXION-V: Los casos adversariales revelaron un error crítico en la condición del bucle (`<` vs `<=`), fallando al buscar en los extremos del arreglo. Una vez corregido este detalle, la implementación se volvió robusta y el pipeline supera todos los casos diseñados al 100%.

IA-REFLEXION-P: Al equipo le queda pendiente identificar que los diccionarios de Python son la estructura nativa que usa este principio. También deben descubrir cómo se redimensiona internamente la tabla cuando hay demasiadas colisiones, tema clave para la Semana 12.